Lab 17

Using Published APIs

WSO2 API Manager 4.7.0 · Practical exercise

Before You Begin

URLs in this guide use placeholders. Replace them with the values of the API Manager you are using:

- **<APIM_HOST>** – host name or IP address of the API Manager server (localhost when API Manager runs on your own laptop).
- **<HTTPS_PORT>** – API Manager HTTPS port: 9443 plus any port offset (Publisher, Developer Portal, Admin Portal, Management Console, token endpoint).
- **<GW_HTTPS_PORT> / <GW_HTTP_PORT>** – API Gateway ports: 8243 (HTTPS) and 8280 (HTTP) plus any port offset.
- **<APIM_HOME>** – the folder where API Manager is extracted.

The servers use a self-signed certificate by default, so the browser shows a security warning the first time: choose Advanced and continue.

Training Objective

Learn how to create, deploy and publish an API, and create an application and a subscription, using the API Manager REST APIs from the command line with cURL (or a REST client such as Postman) instead of the portals.

Business Scenario

PizzaShack wants its delivery team to see the best route to each customer, using a directions API (a mock back end in this lab) exposed through API Manager. Their DevOps team wants to automate API set-up with scripts, so they use the REST APIs of the Publisher and the Developer Portal.

High-Level Steps

- Register a REST API client and get an access token
- Create the API
- Deploy the API to a gateway
- Publish the API
- Create an application
- Subscribe the application to the API and call it

How the REST APIs Are Secured

Every Publisher and Developer Portal REST API call needs an OAuth2 access token with the right scope. You get it in two steps: register a client once (Dynamic Client Registration), then ask the token endpoint for a token with the scopes you need. The scopes used in this lab are:

CC BY 4.0

Scope	Allows
`apim:api_create`	Create APIs, create revisions and deploy them to a gateway
`apim:api_publish`	Change the API lifecycle (for example Publish)
`apim:app_manage`	Create and manage applications in the Developer Portal
`apim:subscribe`	List APIs and applications, and subscribe

Run all commands from the same folder, because the JSON request bodies are read from files with -d @file. Use a plain text editor to create the files.

> In a REST client such as Postman, use the same method, URL, headers and body. For curl `-u user:secret` use **Authorization > Basic Auth**; for `-d @file` paste the file into **Body > raw (JSON)**; for the token calls use **Body > x-www-form-urlencoded**.

Part 1 – Register a Client and Get a Token

1. Open a terminal (Command Prompt on Windows) in an empty folder. Save the API Manager address in a variable, so the commands stay short. Use the same terminal for the whole lab.

macOS / Linux

```
APIM=https://<APIM_HOST>:<HTTPS_PORT>
```

Example – macOS / Linux:

```
APIM=https://localhost:9443
```

Windows (Command Prompt)

```
set APIM=https://<APIM_HOST>:<HTTPS_PORT>
```

Example – Windows (Command Prompt):

```
set APIM=https://localhost:9443
```

2. Create a file named `payload.json` with the content below:

```
{
  "callbackUrl": "www.google.lk",
  "clientName": "rest_api_publisher",
  "owner": "admin",
  "grantType": "password refresh_token",
  "saasApp": true
}
```

3. Register the REST API client. API Manager must be running. `-u admin:admin` sends the admin user name and password; change it if your admin password is different.

macOS / Linux

```
curl -k -X POST -u admin:admin -H "Content-Type: application/json" \
  -d @payload.json \
  "$APIM/client-registration/v0.17/register"
```

CC BY 4.0

Windows (Command Prompt)

```
curl -k -X POST -u admin:admin -H "Content-Type: application/json" ^
  -d @payload.json ^
  "%APIM%/client-registration/v0.17/register"
```

The response contains a clientId and clientSecret. Copy both:

```
> {"clientId":"Wg2sMkl852Jdr8uICj0c","clientName":"rest_api_publisher",
>  "callBackURL":"www.google.lk","clientSecret":"GGdOKRhownFv45AI6gTG",
>  "isSaasApplication":true,"appOwner":"admin", ...}
```

4. Request an access token with the password grant and the `apim:api_create` scope. Replace `<CLIENT_ID>` and `<CLIENT_SECRET>` with the values from step 3:

macOS / Linux

```
curl -k -u "<CLIENT_ID>:<CLIENT_SECRET>" \
  -d "grant_type=password" -d "username=admin" -d "password=admin" \
  -d "scope=apim:api_create" \
  "$APIM/oauth2/token"
```

Example – macOS / Linux:

```
curl -k -u "Wg2sMkl852Jdr8uICj0c:GGdOKRhownFv45AI6gTG" \
  -d "grant_type=password" -d "username=admin" -d "password=admin" \
  -d "scope=apim:api_create" \
  "$APIM/oauth2/token"
```

Windows (Command Prompt)

```
curl -k -u "<CLIENT_ID>:<CLIENT_SECRET>" ^
  -d "grant_type=password" -d "username=admin" -d "password=admin" ^
  -d "scope=apim:api_create" ^
  "%APIM%/oauth2/token"
```

Example – Windows (Command Prompt):

```
curl -k -u "Wg2sMkl852Jdr8uICj0c:GGdOKRhownFv45AI6gTG" ^
  -d "grant_type=password" -d "username=admin" -d "password=admin" ^
  -d "scope=apim:api_create" ^
  "%APIM%/oauth2/token"
```

Copy the access_token from the response:

```
> {"access_token":"169041a9-14fd-3e7c-b5d2-14aa82729bba",
>  "refresh_token":"a49e9e74-cbbb-37db-97de-0bdc74316d49",
>  "scope":"apim:api_create","token_type":"Bearer","expires_in":3600}
```

A token only has the scopes you ask for and is valid for one hour by default. Request a new token when you need a different scope.

CC BY 4.0

Part 2 – Create the API

5. Create a file named `data.json` with the API definition below:

```
{
  "name": "GoogleDirectionsAPI",
  "version": "Beta",
  "context": "/googledirections",
  "policies": ["Unlimited"],
  "endpointConfig": {
    "endpoint_type": "http",
    "sandbox_endpoints": {
      "url": "https://dev-tools.wso2.com/gs/helpers/v1.0/uuid"
    },
    "production_endpoints": {
      "url": "https://dev-tools.wso2.com/gs/helpers/v1.0/uuid"
    }
  }
}
```

On a shared server, add your initials to the name and context (for example `GoogleDirectionsAPI-JD` and `/googledirections-jd`).

6. Create the API. Replace `<ACCESS_TOKEN>` with the token from step 4:

macOS / Linux

```
curl -k -X POST -H "Authorization: Bearer <ACCESS_TOKEN>" \
  -H "Content-Type: application/json" -d @data.json \
  "$APIM/api/am/publisher/v4/apis"
```

Example – macOS / Linux:

```
curl -k -X POST -H "Authorization: Bearer 169041a9-14fd-3e7c-b5d2-14aa82729bba" \
  -H "Content-Type: application/json" -d @data.json \
  "$APIM/api/am/publisher/v4/apis"
```

Windows (Command Prompt)

```
curl -k -X POST -H "Authorization: Bearer <ACCESS_TOKEN>" ^
  -H "Content-Type: application/json" -d @data.json ^
  "%APIM%/api/am/publisher/v4/apis"
```

Example – Windows (Command Prompt):

```
curl -k -X POST -H "Authorization: Bearer 169041a9-14fd-3e7c-b5d2-14aa82729bba" ^
  -H "Content-Type: application/json" -d @data.json ^
  "%APIM%/api/am/publisher/v4/apis"
```

The response (HTTP 201) describes the new API. Copy its `id`. The lifecycle status is CREATED and the API has the default `/*` resources (GET, PUT, POST, DELETE, PATCH).

```
> {"id":"c10a952d-9648-4160-96fe-54a96f5c7080","name":"GoogleDirectionsAPI",
>  "context":"/googledirections","version":"Beta","provider":"admin",
>  "lifeCycleStatus":"CREATED", ...}
```

7. Save the API ID in a variable:

macOS / Linux

```
API_ID=<API_ID>
```

CC BY 4.0

Example – macOS / Linux:

```
API_ID=c10a952d-9648-4160-96fe-54a96f5c7080
```

Windows (Command Prompt)

```
set API_ID=<API_ID>
```

Example – Windows (Command Prompt):

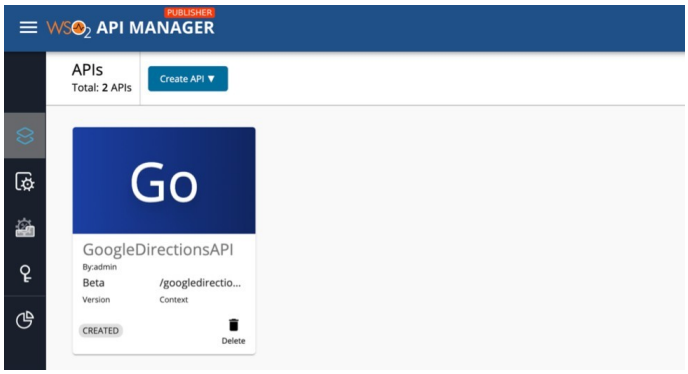
```
set API_ID=c10a952d-9648-4160-96fe-54a96f5c7080
```

8. Open the Publisher and refresh the page. GoogleDirectionsAPI is listed:

```
https://<APIM_HOST>:<HTTPS_PORT>/publisher
```

Example:

```
https://localhost:9443/publisher
```



Part 3 – Deploy the API

An API must be deployed to a gateway before it can be called. First create a revision (a snapshot of the API), then deploy the revision. The token from step 4 (apim:api_create) is enough for both calls while the API is still in the CREATED state. Once an API is published, creating a new revision also needs the apim:api_publish scope (otherwise: Insufficient permission).

9. Replace the content of `data.json` with:

```
{
  "description": "API Revision 1"
}
```

10. Create the revision. Replace the access token:

macOS / Linux

```
curl -k -X POST -H "Authorization: Bearer <ACCESS_TOKEN>" \
  -H "Content-Type: application/json" -d @data.json \
  "$APIM/api/am/publisher/v4/apis/$API_ID/revisions"
```

CC BY 4.0

Example – macOS / Linux:

```
curl -k -X POST -H "Authorization: Bearer 169041a9-14fd-3e7c-b5d2-14aa82729bba" \
  -H "Content-Type: application/json" -d @data.json \
  "$APIM/api/am/publisher/v4/apis/$API_ID/revisions"
```

Windows (Command Prompt)

```
curl -k -X POST -H "Authorization: Bearer <ACCESS_TOKEN>" ^
  -H "Content-Type: application/json" -d @data.json ^
  "%APIM%/api/am/publisher/v4/apis/%API_ID%/revisions"
```

Example – Windows (Command Prompt):

```
curl -k -X POST -H "Authorization: Bearer 169041a9-14fd-3e7c-b5d2-14aa82729bba" ^
  -H "Content-Type: application/json" -d @data.json ^
  "%APIM%/api/am/publisher/v4/apis/%API_ID%/revisions"
```

Copy the revision `id` from the response (HTTP 201). This is the `<REVISION_ID>`:

```
> {"displayName":"Revision 1","id":"a83434fc-3309-4583-a781-2c67f18861c2",
>  "description":"API Revision 1","createdTime":1620103889000,
>  "apiInfo":{"id":"c10a952d-9648-4160-96fe-54a96f5c7080"},"deploymentInfo":[]}
```

11. Save the revision ID in a variable:

macOS / Linux

```
REV_ID=<REVISION_ID>
```

Example – macOS / Linux:

```
REV_ID=a83434fc-3309-4583-a781-2c67f18861c2
```

Windows (Command Prompt)

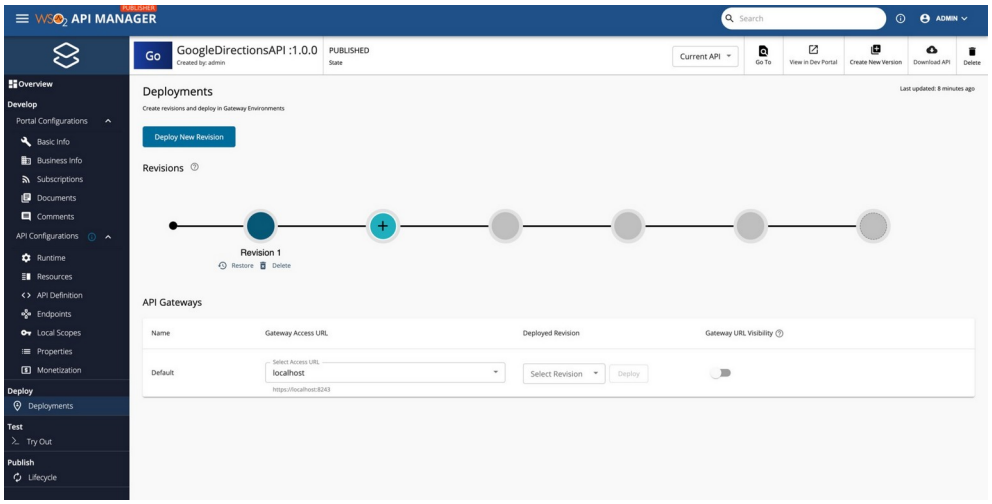
```
set REV_ID=<REVISION_ID>
```

Example – Windows (Command Prompt):

```
set REV_ID=a83434fc-3309-4583-a781-2c67f18861c2
```

CC BY 4.0

12. In the Publisher, open the API and go to **Deploy > Deployments**. Revision 1 is shown but not deployed yet:



13. Replace the content of `data.json` with the gateway environment to deploy to:

```
[
  {
    "name": "Default",
    "vhost": "localhost",
    "displayOnDevportal": true
  }
]
```

14. Deploy the revision to the Default gateway. Replace the access token:

macOS / Linux

```
curl -k -X POST -H "Authorization: Bearer <ACCESS_TOKEN>" \
  -H "Content-Type: application/json" -d @data.json \
  "$APIM/api/am/publisher/v4/apis/$API_ID/deploy-revision?revisionId=$REV_ID"
```

Example – macOS / Linux:

```
curl -k -X POST -H "Authorization: Bearer 169041a9-14fd-3e7c-b5d2-14aa82729bba" \
  -H "Content-Type: application/json" -d @data.json \
  "$APIM/api/am/publisher/v4/apis/$API_ID/deploy-revision?revisionId=$REV_ID"
```

Windows (Command Prompt)

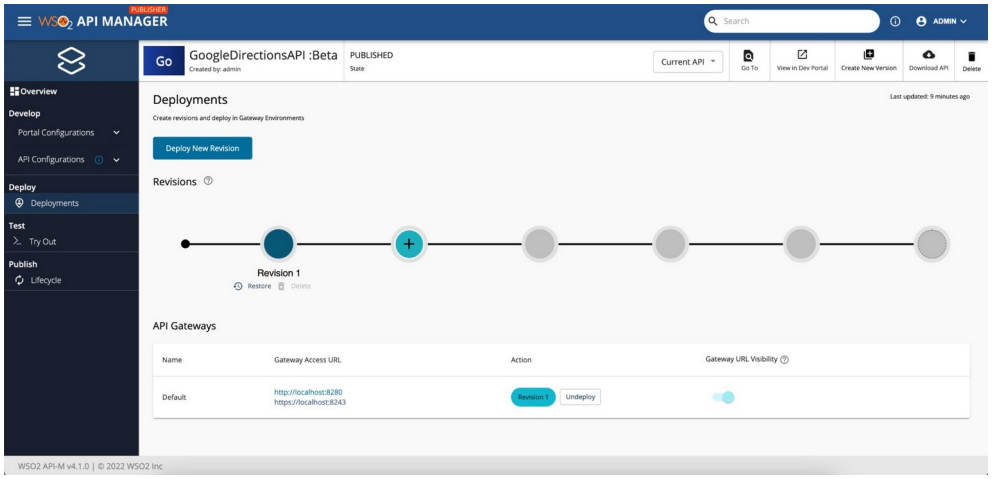
```
curl -k -X POST -H "Authorization: Bearer <ACCESS_TOKEN>" ^
  -H "Content-Type: application/json" -d @data.json ^
  "%APIM%/api/am/publisher/v4/apis/%API_ID%/deploy-revision?revisionId=%REV_ID%"
```

Example – Windows (Command Prompt):

```
curl -k -X POST -H "Authorization: Bearer 169041a9-14fd-3e7c-b5d2-14aa82729bba" ^
  -H "Content-Type: application/json" -d @data.json ^
  "%APIM%/api/am/publisher/v4/apis/%API_ID%/deploy-revision?revisionId=%REV_ID%"
```

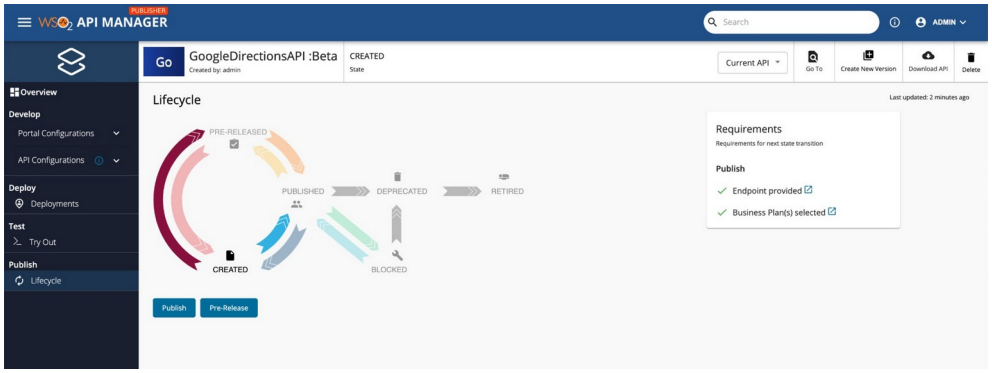
CC BY 4.0

15. Refresh the Deployments page. The API is now deployed to the **Default** gateway:



Part 4 – Publish the API

16. In the Publisher, open GoogleDirectionsAPI and go to **Lifecycle**. The state is **CREATED**:



17. Publishing needs a token with the `apim:api_publish` scope. Request it with the same client ID and secret:

macOS / Linux

```
curl -k -u "<CLIENT_ID>:<CLIENT_SECRET>" \
  -d "grant_type=password" -d "username=admin" -d "password=admin" \
  -d "scope=apim:api_publish" \
  "$APIM/oauth2/token"
```

Example – macOS / Linux:

```
curl -k -u "Wg2sMkl852Jdr8uICj0c:GGdOKRhownFv45AI6gTG" \
  -d "grant_type=password" -d "username=admin" -d "password=admin" \
  -d "scope=apim:api_publish" \
  "$APIM/oauth2/token"
```

Windows (Command Prompt)

```
curl -k -u "<CLIENT_ID>:<CLIENT_SECRET>" ^
  -d "grant_type=password" -d "username=admin" -d "password=admin" ^
  -d "scope=apim:api_publish" ^
  "%APIM%/oauth2/token"
```

CC BY 4.0

Example – Windows (Command Prompt):

```
curl -k -u "Wg2sMkl852Jdr8uICj0c:GGdOKRhownFv45AI6gTG" ^
  -d "grant_type=password" -d "username=admin" -d "password=admin" ^
  -d "scope=apim:api_publish" ^
  "%APIM%/oauth2/token"
```

18. Publish the API with the new token. Replace the access token:

macOS / Linux

```
curl -k -X POST -H "Authorization: Bearer <ACCESS_TOKEN>" \
  "$APIM/api/am/publisher/v4/apis/change-lifecycle?apiId=$API_ID&action=Publish"
```

Example – macOS / Linux:

```
curl -k -X POST -H "Authorization: Bearer 169041a9-14fd-3e7c-b5d2-14aa82729bba" \
  "$APIM/api/am/publisher/v4/apis/change-lifecycle?apiId=$API_ID&action=Publish"
```

Windows (Command Prompt)

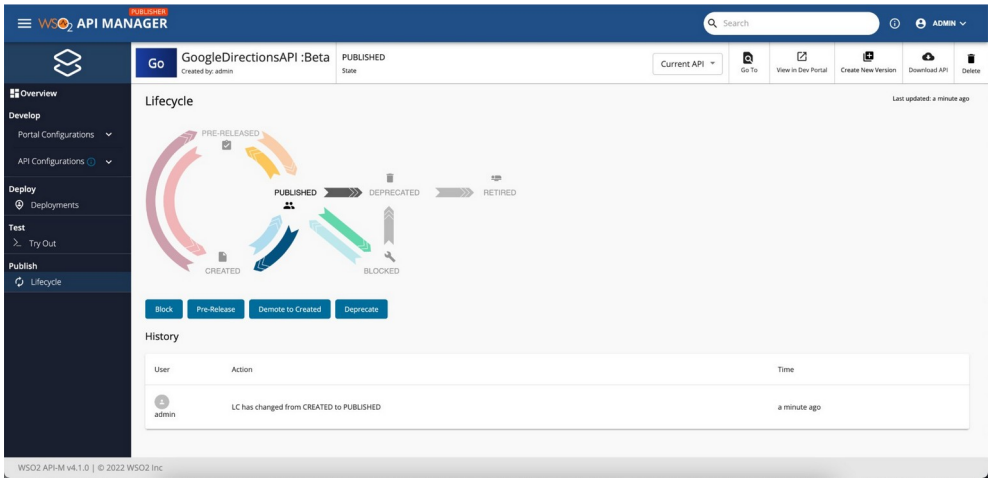
```
curl -k -X POST -H "Authorization: Bearer <ACCESS_TOKEN>" ^
  "%APIM%/api/am/publisher/v4/apis/change-lifecycle?apiId=%API_ID%&action=Publish"
```

Example – Windows (Command Prompt):

```
curl -k -X POST -H "Authorization: Bearer 169041a9-14fd-3e7c-b5d2-14aa82729bba" ^
  "%APIM%/api/am/publisher/v4/apis/change-lifecycle?apiId=%API_ID%&action=Publish"
```

The response is HTTP 200 with `"lifecycleState":{"state":"Published", ...}`.

19. Refresh the Lifecycle page in the Publisher. The state is now PUBLISHED:



Part 5 – Create an Application

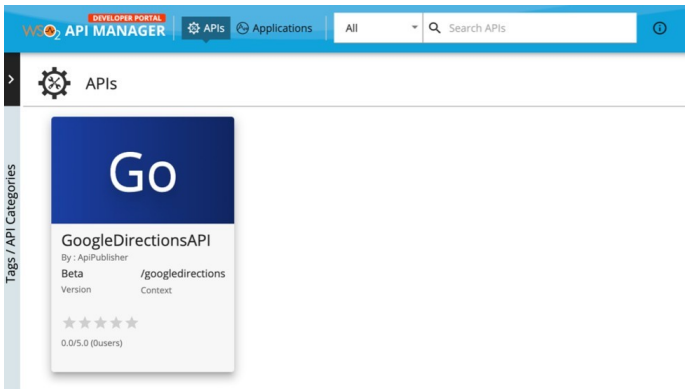
20. Log in to the Developer Portal as admin. GoogleDirectionsAPI is visible:

```
https://<APIM_HOST>:<HTTPS_PORT>/devportal
```

Example:

```
https://localhost:9443/devportal
```

CC BY 4.0



21. In `payload.json`, change the clientName to `rest_api_devportal` and save:

```
{
  "callbackUrl": "www.google.lk",
  "clientName": "rest_api_devportal",
  "owner": "admin",
  "grantType": "password refresh_token",
  "saasApp": true
}
```

22. Register this client, as in step 3, and copy the new clientId and clientSecret:

macOS / Linux

```
curl -k -X POST -u admin:admin -H "Content-Type: application/json" \
  -d @payload.json \
  "$APIM/client-registration/v0.17/register"
```

Windows (Command Prompt)

```
curl -k -X POST -u admin:admin -H "Content-Type: application/json" ^
  -d @payload.json ^
  "%APIM%/client-registration/v0.17/register"
```

23. Request a token with the `apim:subscribe` and `apim:app_manage` scopes, using the new client ID and secret:

macOS / Linux

```
curl -k -u "<CLIENT_ID>:<CLIENT_SECRET>" \
  -d "grant_type=password" -d "username=admin" -d "password=admin" \
  -d "scope=apim:subscribe apim:app_manage" \
  "$APIM/oauth2/token"
```

Example – macOS / Linux:

```
curl -k -u "Wg2sMkl852Jdr8uICj0c:GGdOKRhownFv45AI6gTG" \
  -d "grant_type=password" -d "username=admin" -d "password=admin" \
  -d "scope=apim:subscribe apim:app_manage" \
  "$APIM/oauth2/token"
```

Windows (Command Prompt)

```
curl -k -u "<CLIENT_ID>:<CLIENT_SECRET>" ^
  -d "grant_type=password" -d "username=admin" -d "password=admin" ^
  -d "scope=apim:subscribe apim:app_manage" ^
  "%APIM%/oauth2/token"
```

CC BY 4.0

Example – Windows (Command Prompt):

```
curl -k -u "Wg2sMkl852Jdr8uICj0c:GGdOKRhownFv45AI6gTG" ^
  -d "grant_type=password" -d "username=admin" -d "password=admin" ^
  -d "scope=apim:subscribe apim:app_manage" ^
  "%APIM%/oauth2/token"
```

The scope field of the response shows both scopes:

```
> {"access_token":"68b17d4e-9466-3e92-a809-8ce2ea6f7b92",
>  "scope":"apim:app_manage apim:subscribe",
>  "token_type":"Bearer","expires_in":3600}
```

24. Replace the content of `data.json` with the application details:

```
{
  "name": "GoogleDirectionsApp",
  "throttlingPolicy": "Unlimited",
  "description": "GoogleDirectionsAPI App",
  "tokenType": "JWT"
}
```

25. Create the application. Replace the access token with the one from step 23:

macOS / Linux

```
curl -k -X POST -H "Authorization: Bearer <ACCESS_TOKEN>" \
  -H "Content-Type: application/json" -d @data.json \
  "$APIM/api/am/devportal/v3/applications"
```

Example – macOS / Linux:

```
curl -k -X POST -H "Authorization: Bearer 169041a9-14fd-3e7c-b5d2-14aa82729bba" \
  -H "Content-Type: application/json" -d @data.json \
  "$APIM/api/am/devportal/v3/applications"
```

Windows (Command Prompt)

```
curl -k -X POST -H "Authorization: Bearer <ACCESS_TOKEN>" ^
  -H "Content-Type: application/json" -d @data.json ^
  "%APIM%/api/am/devportal/v3/applications"
```

Example – Windows (Command Prompt):

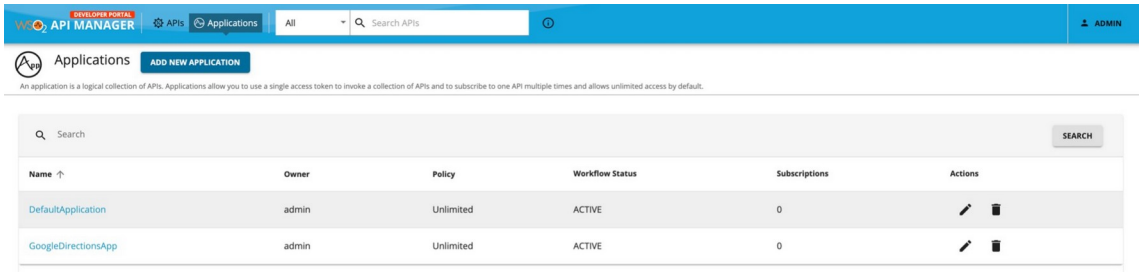
```
curl -k -X POST -H "Authorization: Bearer 169041a9-14fd-3e7c-b5d2-14aa82729bba" ^
  -H "Content-Type: application/json" -d @data.json ^
  "%APIM%/api/am/devportal/v3/applications"
```

The response (HTTP 201) contains the `applicationId`:

```
> {"applicationId":"b3560453-2e3e-413e-a42b-edad3522953e",
>  "name":"GoogleDirectionsApp","throttlingPolicy":"Unlimited",
>  "tokenType":"JWT","status":"APPROVED","subscriptionCount":0, ...}
```

CC BY 4.0

26. In the Developer Portal, go to **Applications**. GoogleDirectionsApp is listed:



Part 6 – Subscribe and Call the API

The token from step 23 also has the apim:subscribe scope, so you can use it for this part.

27. List your applications and copy the applicationId of GoogleDirectionsApp:

macOS / Linux

```
curl -k -H "Authorization: Bearer <ACCESS_TOKEN>" \
  "$APIM/api/am/devportal/v3/applications"
```

Example – macOS / Linux:

```
curl -k -H "Authorization: Bearer 169041a9-14fd-3e7c-b5d2-14aa82729bba" \
  "$APIM/api/am/devportal/v3/applications"
```

Windows (Command Prompt)

```
curl -k -H "Authorization: Bearer <ACCESS_TOKEN>" ^
  "%APIM%/api/am/devportal/v3/applications"
```

Example – Windows (Command Prompt):

```
curl -k -H "Authorization: Bearer 169041a9-14fd-3e7c-b5d2-14aa82729bba" ^
  "%APIM%/api/am/devportal/v3/applications"
```

```
> {"count":2,"list":[{"applicationId":"d7e7c965-6b42-4937-a5f0-8c6dfaac2923",
>  "name":"DefaultApplication", ...},{"applicationId":"b3560453-2e3e-413e-a42b-
>  edad3522953e","name":"GoogleDirectionsApp", ...}], ...}
```

28. List the published APIs and copy the id of GoogleDirectionsAPI:

macOS / Linux

```
curl -k -H "Authorization: Bearer <ACCESS_TOKEN>" \
  "$APIM/api/am/devportal/v3/apis"
```

Example – macOS / Linux:

```
curl -k -H "Authorization: Bearer 169041a9-14fd-3e7c-b5d2-14aa82729bba" \
  "$APIM/api/am/devportal/v3/apis"
```

Windows (Command Prompt)

```
curl -k -H "Authorization: Bearer <ACCESS_TOKEN>" ^
  "%APIM%/api/am/devportal/v3/apis"
```

CC BY 4.0

Example – Windows (Command Prompt):

```
curl -k -H "Authorization: Bearer 169041a9-14fd-3e7c-b5d2-14aa82729bba" ^
  "%APIM%/api/am/devportal/v3/apis"
```

```
> {"count":1,"list":[{"id":"c10a952d-9648-4160-96fe-54a96f5c7080",
>  "name":"GoogleDirectionsAPI","context":"/googledirections","version":"Beta",
>  "lifeCycleStatus":"PUBLISHED", ...}], ...}
```

29. Replace the content of `data.json` with the subscription. Put in the API id and applicationId you copied:

```
{
  "apiId": "<API_ID>",
  "applicationId": "<APPLICATION_ID>",
  "throttlingPolicy": "Unlimited"
}
```

30. Create the subscription:

macOS / Linux

```
curl -k -X POST -H "Authorization: Bearer <ACCESS_TOKEN>" \
  -H "Content-Type: application/json" -d @data.json \
  "$APIM/api/am/devportal/v3/subscriptions"
```

Example – macOS / Linux:

```
curl -k -X POST -H "Authorization: Bearer 169041a9-14fd-3e7c-b5d2-14aa82729bba" \
  -H "Content-Type: application/json" -d @data.json \
  "$APIM/api/am/devportal/v3/subscriptions"
```

Windows (Command Prompt)

```
curl -k -X POST -H "Authorization: Bearer <ACCESS_TOKEN>" ^
  -H "Content-Type: application/json" -d @data.json ^
  "%APIM%/api/am/devportal/v3/subscriptions"
```

Example – Windows (Command Prompt):

```
curl -k -X POST -H "Authorization: Bearer 169041a9-14fd-3e7c-b5d2-14aa82729bba" ^
  -H "Content-Type: application/json" -d @data.json ^
  "%APIM%/api/am/devportal/v3/subscriptions"
```

The response (HTTP 201) shows the new subscription:

```
> {"subscriptionId":"7dbe818b-9729-4c24-be59-7e0f7b9ee7a5",
>  "applicationId":"b3560453-...","apiId":"c10a952d-...",
>  "throttlingPolicy":"Unlimited","status":"UNBLOCKED", ...}
```

CC BY 4.0

31. In the Developer Portal, open **Applications > GoogleDirectionsApp > Subscriptions**. GoogleDirectionsAPI is listed with the Unlimited business plan.

32. Open **Production Keys**, click **Generate Keys**, then **Generate Access Token** and copy the token.

33. Call the API through the gateway with this token:

    macOS / Linux

```
curl -k -H "Authorization: Bearer <ACCESS_TOKEN>" \
  "https://<APIM_HOST>:<GW_HTTPS_PORT>/googledirections/Beta"
```

    Example – macOS / Linux:

```
curl -k -H "Authorization: Bearer 169041a9-14fd-3e7c-b5d2-14aa82729bba" \
  "https://localhost:8243/googledirections/Beta"
```

    Windows (Command Prompt)

```
curl -k -H "Authorization: Bearer <ACCESS_TOKEN>" ^
  "https://<APIM_HOST>:<GW_HTTPS_PORT>/googledirections/Beta"
```

    Example – Windows (Command Prompt):

```
curl -k -H "Authorization: Bearer 169041a9-14fd-3e7c-b5d2-14aa82729bba" ^
  "https://localhost:8243/googledirections/Beta"
```

    The mock back end answers with a random ID:

```
> {"uuid":"d3e06487-eb6c-48f9-b815-88c9dbf4bf05"}
```

Expected Outcome

You created, deployed and published an API, created an application and subscribed it to the API using the API Manager REST APIs with cURL (or a REST client) instead of the Publisher and Developer Portal, and called the API through the gateway.

CC BY 4.0